代理走到了缺陷、也触发了它，却认不出来：编码代理 Reach 93.7% → Detect 51.4%，GUI 专用代理 Detect 4.2%

探索性测试没有脚本：代理要自己进到相关页面（**Reach**）、做出让缺陷显现的操作（**Trigger**）、再认出并报告异常（**Detect**）。现有评测只看终态、只认预先标注的缺陷，三步的失败混在一起，界面显示类缺陷也基本不在考察范围。作者从真实 Android 应用的 issue 构建可重置场景，并用经人工校验的评审器 GUIJudge 把三步分开打分。

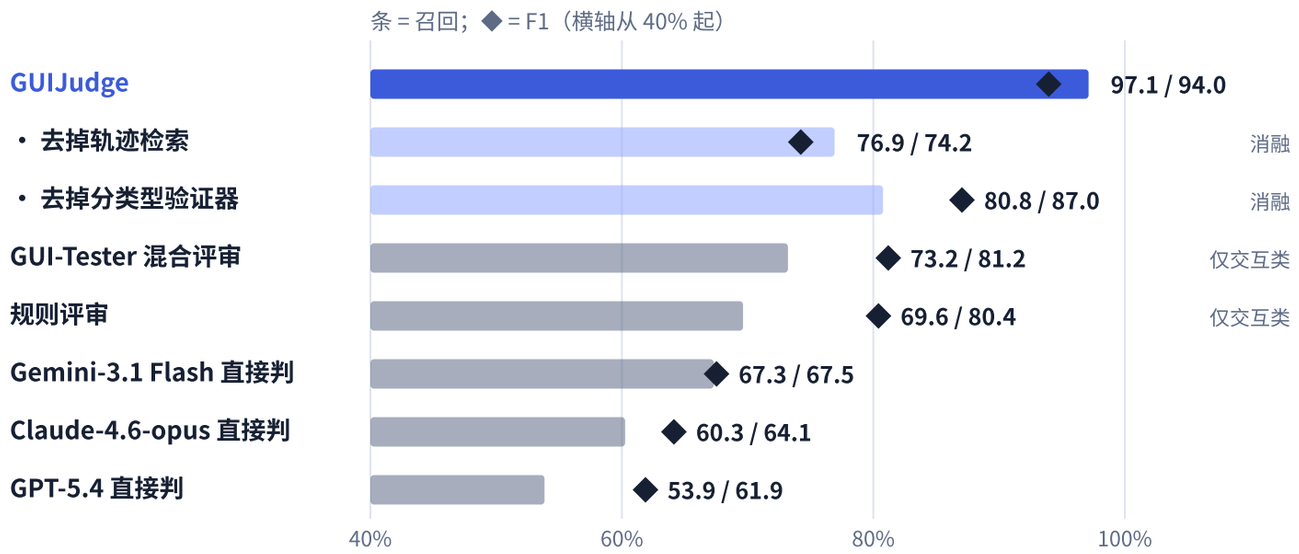
01 基准怎么搭　§3、App. A

508	61	82	384
真实缺陷（公开 issue / bug 报告）	个 Android 应用	个可重置场景	条多入口导航任务



每个场景配一份**测试依据**：从开发者原始描述中整理出的前置条件、交互/事件、可观察证据，不写具体缺陷、不规定唯一路径。导航任务从不同入口、不同提示程度逼近同一功能，不透露缺陷。

02 先校验评审器　RQ1 · 人工参考集上的 Detect 判定 · Tab.1、App. B–C



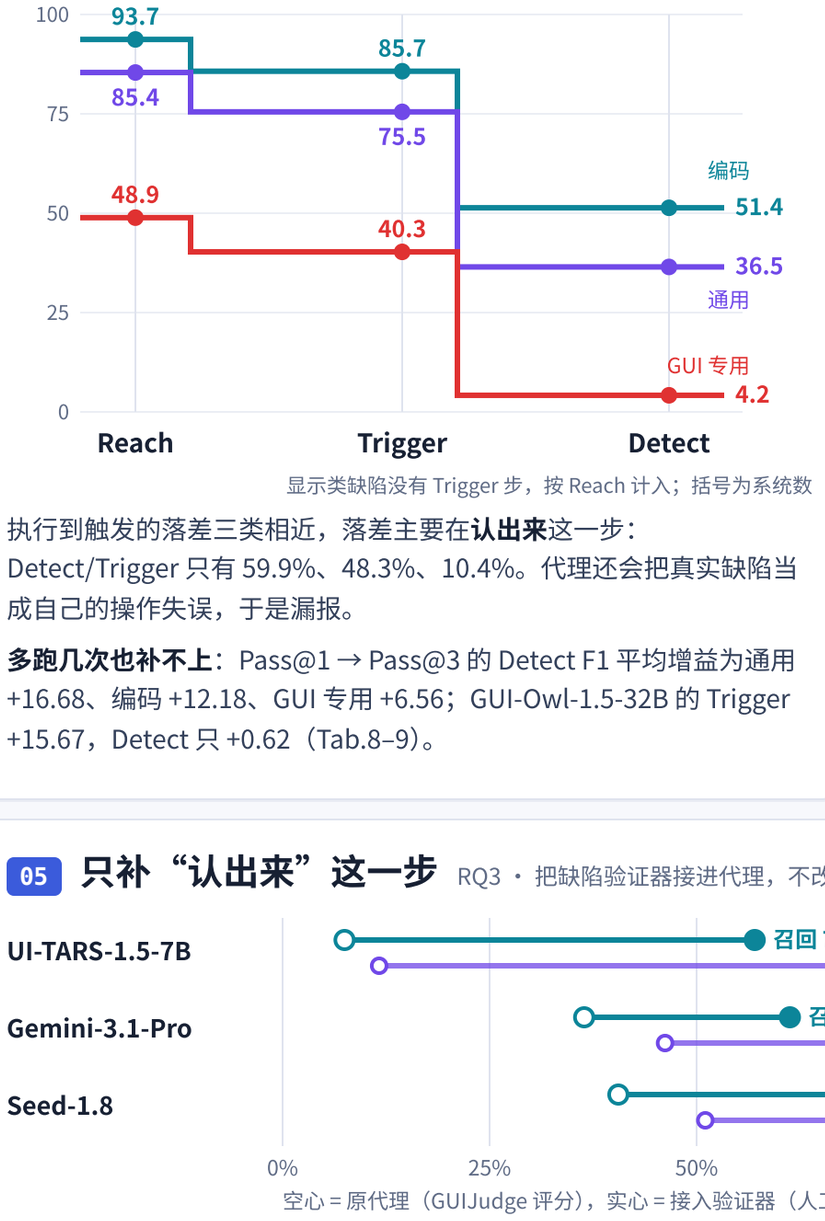
两名有 Android 测试经验的标注者独立标注，看不到 GUIJudge 输出，κ = 0.95 / 0.94 / 0.90（Reach / Trigger / Detect）。拿掉轨迹检索 Detect 召回降到 76.9%，换成统一验证器降到 80.8%。**去掉轨迹证据只留测试依据和代理报告，正向 Detect 判定降为 0**，测试依据本身不是答案。换 3 个其他骨干，判定两两 κ 0.82–0.91。

参考集规模正文未写明；由一致率 98.51/97.03/95.05% 反推约 202 条轨迹（我们的推算）

GUIJudge 怎么把三步分开判　§4、App. B.1 · 示意重绘



03 三步逐级流失　Pass@1 召回 · 各类系统平均 · §5.3、Tab.8



显示类缺陷没有 Trigger 步，按 Reach 计入；括号为系统数

执行到触发的落差三类相近，落差主要在**认出来**这一步：Detect/Trigger 只有 59.9%、48.3%、10.4%。代理还会把真实缺陷当成自己的操作失误，于是漏报。

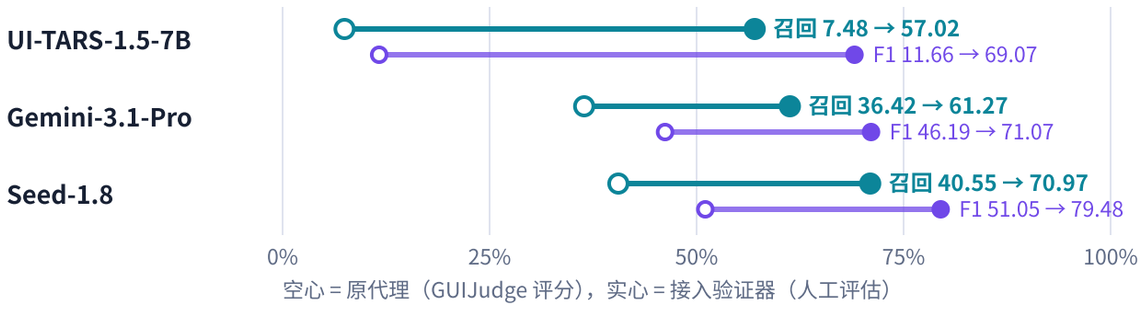
多跑几次也补不上：Pass@1 → Pass@3 的 Detect F1 平均增益为通用 +16.68、编码 +12.18、GUI 专用 +6.56；GUI-Owl-1.5-32B 的 Trigger +15.67，Detect 只 +0.62（Tab.8–9）。

04 哪类缺陷最难认　Pass@1 Detect F1（%）· 13 个系统 × 5 种故障 · Tab.2

		显示类		交互类			
		CR	EL	NLE	ONR	UTR	总体
编码代理	Claude Code（Claude）	70.8	69.8	75.2	62.3	60.8	69.4
	Codex（GPT-6-Astra）	74.5	64.2	70.5	67.7	56.3	65.9
	Claude Code（DeepSeek）	58.3	48.8	69.3	63.3	47.6	55.2
通用模型	Claude-5.5-Opus	63.1	56.8	64.7	57.8	57.6	59.3
	GPT-6-Astra	65.7	59.4	59.3	62.3	39.2	56.8
	Seed-1.8	65.3	56.3	48.1	45.2	35.7	51.0
	Gemini-3.1-Pro	59.7	43.0	55.2	46.6	33.9	46.2
	DeepSeek-4.1-Flash	57.8	43.9	20.3	32.6	19.7	38.5
	Qwen-3-VL-Plus	42.6	33.6	37.4	33.9	20.5	32.8
GUI 专用	UI-TARS-1.5-7B	9.7	3.3	37.9	10.3	7.7	11.7
	MAI-UI-8B	2.1	2.3	14.3	15.4	6.1	7.6
	GUI-Owl-1.5-32B	17.6	6.6	8.7	4.8	0.0	7.3
	GUI-Owl-1.5-8B	2.1	3.4	3.0	0.0	1.8	2.3

显示类里，元素布局 EL 比内容渲染 CR 难（13 个系统里 11 个 CR 更高）：判断“布局是否合理”要知道该应用本来长什么样。交互类里，**需要跨状态核对结果的 UTR 对全部 13 个系统都比 NLE 难**。

05 只补“认出来”这一步　RQ3 · 把缺陷验证器接进代理，不改导航策略 · 人工评估 · Tab.3



空心 = 原代理（GUIJudge 评分），实心 = 接入验证器（人工评估）

UI-TARS-1.5-7B 召回 7.5% → 57.0%。这是给代理**加了一个由强模型驱动的验证模块**后的系统表现，不是模型本身变会认了；作者也在限制里这样写。

06 为什么编码代理更好　§5.3、App. C.5 · 同骨干套上 Claude Code / Codex

三个编码代理配置都比自己的基础模型强，Detect F1 增益 +9.04 到 +16.64，而 Reach、Trigger 只增 0.59–6.60。作者从轨迹里看到三种差别：

主动再看一次
基础模型把加载中的画面当显示缺陷；编码代理等一下再截图，确认内容正常渲染。

先怀疑自己
误点开了别的帖子，基础模型报导航缺陷；编码代理回看上一步、重试，确认导航正常。

记住目标再比对
两个骨干都去查下载列表；DeepSeek 把无关下载当成功，Claude 对照上下文里的目标发现不符。

同一框架下 Claude 与 DeepSeek 的 Detect F1 差 14.27（Pass@3 时 14.87），Trigger 却相近：harness 改善取证，**认缺陷仍取决于底座模型**。

限制与证据边界

- 只有 Android，Web 与桌面没有实证；测试依据部分依赖人工整理；Pass@1 为单次运行，未报告区间。
- RQ2 的全部分数由 GUIJudge（默认骨干 Claude-4.6-Opus）给出；这个默认骨干是在**同一份人工参考集**上按表现选出的（App. B.3）。排名前列的 Claude Code（Claude-5.5-Opus）与评审同属一个模型家族，RQ2 没有用其他骨干重评。
- 开放集指评审可以认可预设之外的真实缺陷；但这类“新发现”是否被正确判定，没有单独的人工正负核验。
- RQ3 的对比口径不一：接入前的数值取自 Tab.2（GUIJudge 评分），接入后来自人工评估；人工评估者人数与一致性未说明；验证器带来额外模型调用，未与等算力基线比较。

和此前画报的关系　我们的阅读

本期另一篇入选的 2610.39282 发现“加载未完成”会让点击落空；这里的编码代理案例 1 则显示，基础模型会把**加载中的画面误报为显示缺陷**——两篇都指向同一个工程事实：截图的时刻本身就是证据的一部分。

我们的阅读：本文最有用的是把一个总分拆成三步并给出可核查的评审器，使“GUI 代理不会测试”变成“会走、会点、不会认”。GUI 专用小模型的 Reach 只有 25.6–70.1%，导航仍是它们的第一道坎；对通用模型与编码代理，提升空间几乎都在 Detect。